



HUMAN COMPUTER INTERACTION

Content

Introduction

Tools used

Voice and Gesture system

UI design and connection

Testing and evaluation

Meet the team



AYOMIPOSİ ADEBAYO
AI ENGINEER



HAFAEEZ AHMED
SOFTWARE ENGINEER



FIYINFOLUWA AKINBO
SOFTWARE ENGINEER



WAEŁ ALLİL
SOFTWARE ENGINEER



BIMBO ADETUNJI
COMPUTER ENGINEER

INTRODUCTION

This project presents the design and implementation of a multimodal human-computer interaction system that enables users to control a computer using natural voice commands and hand gestures. The system integrates real-time speech recognition, gesture detection, and intent-based action execution within a unified desktop application.

Goal of the Project:

The primary goal of this project is to develop an intuitive, hands-free interaction system that improves accessibility, usability, and efficiency by allowing users to interact with computing systems using natural human inputs rather than traditional devices such as keyboards and mice.

TOOLS USED

Backend

- Python
- OpenCV
- MediaPipe
- PyAudio
- Picovoice Porcupine
- AssemblyAI API
- OpenAI GPT-4.1 API
- NumPy
- OS Automation Libraries

Frontend

- React.js
- Vite
- JavaScript (ES6+)
- HTML5
- CSS3
- Lucide React Icons

System Integration & Communication

- Electron.js
- Node.js
- Child Process API
- IPC (Inter-Process Communication)
- JSON

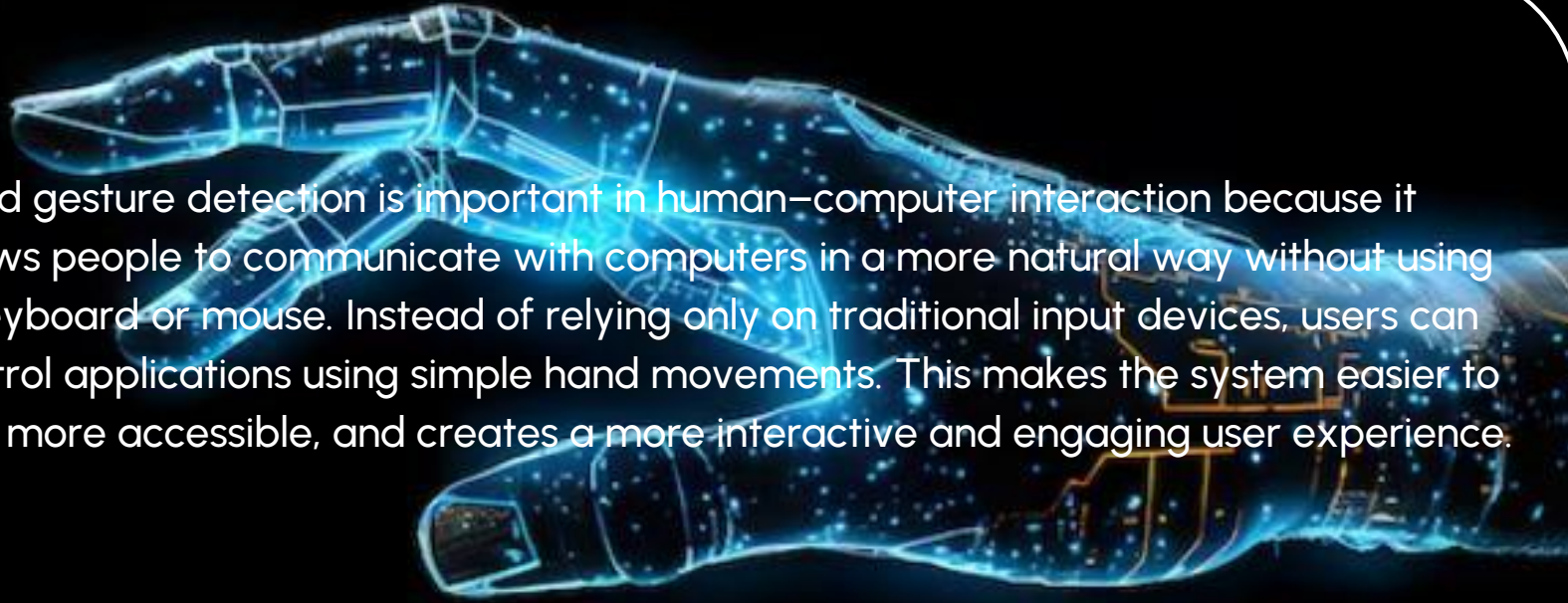
VOICE SYSTEM

Firstly we will look at the implementation and nature of how the voice interaction operates

This includes a system that is fully automated and works with continuous interactivity that prioritises accuracy and flexibility

The system works by transcribing user input which transcribes the audio and into text and uses a LLM to analyse it, which maps it to the correct intent in the dispatcher and triggers the action.

GESTURE SYSTEM



Hand gesture detection is important in human–computer interaction because it allows people to communicate with computers in a more natural way without using a keyboard or mouse. Instead of relying only on traditional input devices, users can control applications using simple hand movements. This makes the system easier to use, more accessible, and creates a more interactive and engaging user experience.

GESTURE SYSTEM

The gesture functionality works by detecting points on the hands which is then used to calculate or determine the gesture held up based on the positions of the points from each other (same as motion). After detection the intent that matches the gesture, it will get triggered from the despatch same as the voice action but with the assistance of other tools also



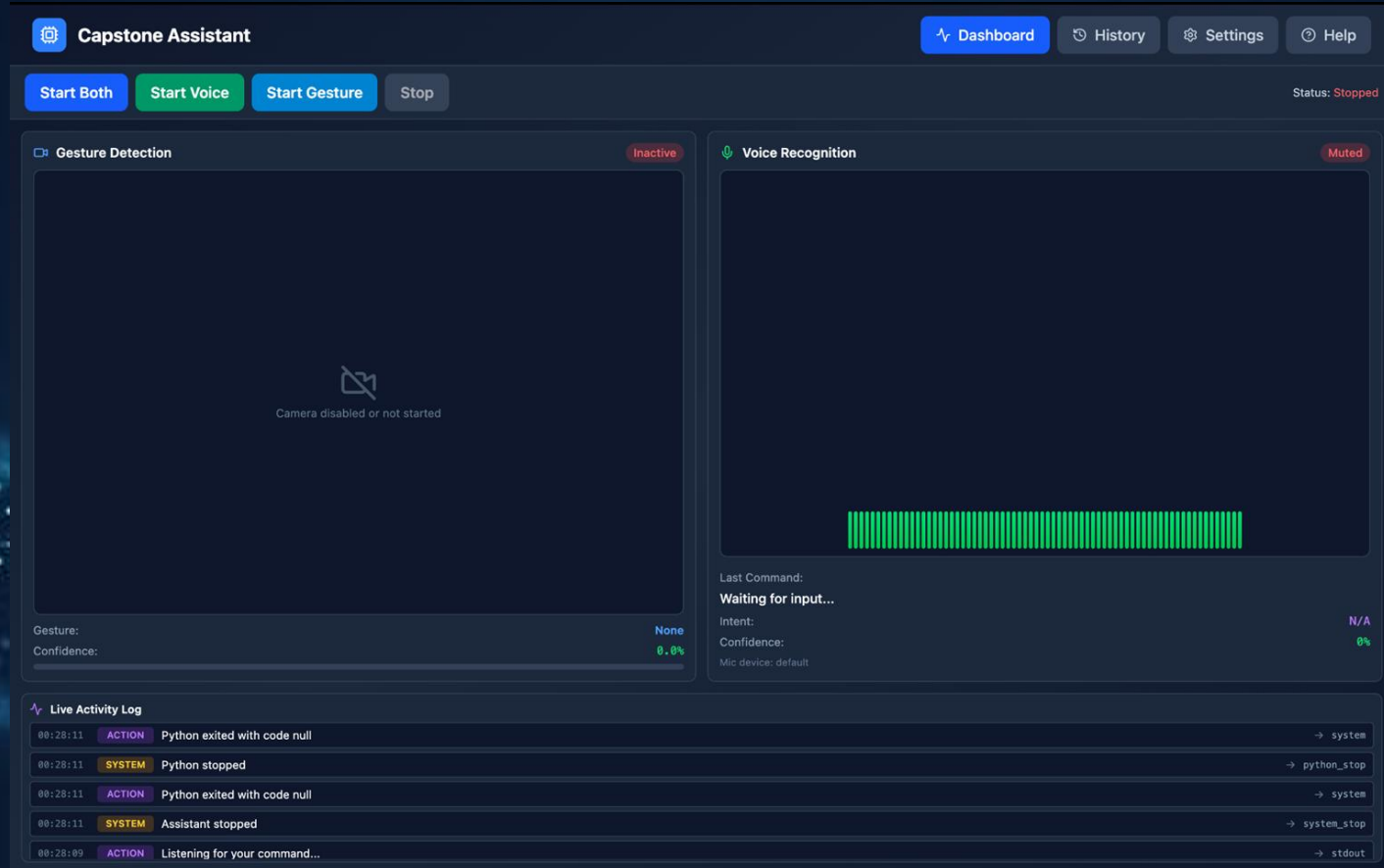
UI DESIGN

The user interface serves as the primary interaction point between the user and the multimodal system. It enables intuitive control, real-time feedback, and seamless coordination between voice and gesture inputs.

User Interface Overview

- Provides a central dashboard to control the system
- Users can start voice, gesture, or combined interaction modes
- Displays live camera feed and microphone status
- Shows detected gestures, voice commands, and confidence levels
- Includes a real-time activity log for system feedback
- Designed to be simple, visual, and easy to use

UI DESIGN



FRONT AND BACKEND CONNECTION

The front-end and back-end connection enables communication between the user interface and the system's core processing modules. This connection ensures that user actions are correctly transmitted, processed, and reflected in real time through system responses and logs.

Front-End and Back-End Connection Overview

- Establishes communication between the Electron/React interface and the Python processing engine
- Uses a child process mechanism to start, stop, and control Python modules from the UI
- Allows the front end to send execution modes (voice, gesture, or both) to the back end
- Receives real-time logs, detected actions, and system feedback from Python processes
- Ensures synchronized operation between user interactions and backend execution
- Enables modular, scalable integration between interface and processing logic

TESTING AND EVALUATION

Testing and Evaluation

- Adopted a trial-and-error testing approach to iteratively refine system behavior and accuracy
- Tested multiple speech recognition engines (e.g., Whisper, external APIs) and replaced less accurate options
- Evaluated fuzzy matching techniques for intent recognition and removed them due to inconsistent results
- Continuously tested gesture recognition accuracy under different lighting, distances, and hand positions
- Modified system architecture from press-to-speak to a fully automatic wake-word-based voice interaction
- Tested different runtime configurations for voice-only, gesture-only, and combined modes
- Validated real-time responsiveness by monitoring delays between detection and action execution
- Used live system logs and visual feedback to identify errors, misclassifications, and performance bottlenecks
- Improved accuracy through repeated calibration, confidence thresholds, and temporal filtering



THANK YOU